Problem Set 4

1.(a)

If an action sequence is a solution for a belief state, it guarantees reaching the goal from every state in the belief state. When we take any smaller set of states from it, those states are still ones where the sequence leads to the goal successfully. Thus, the action sequence also serves as a valid solution for any subset of the original belief state.

1.(b)

If an action sequence is not a solution for a belief state, it can't be a solution to any superset of b in (a). If the sequence does not reach the goal from state in original belief, that state will still be present in any superset, and it doesn't work.

1.(c)

I can take advantage of my answer a and b by modifying graph search for sensorless problems. If an action sequence is a solution for a belief state, then it is also a solution for any subset of that belief state, which means if we encounter a belief state that is a subset of one which already solved during search, it can be solved without exploring, which reduce computation.

If a belief state failed like no action sequence leads to a goal for all states, Any superset must fail as well. Thus, encountering a superset of failed state let me prune search space.

1.(d)

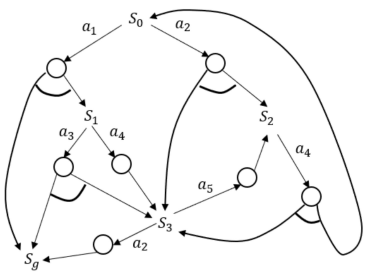
And-node: the environment "chooses" to have an outcome for each action.

Or-node: the agent chooses an action.

In partially observable environments, similar like answer (c), I can prune the subsets of a belief state which solution is already founded and order the belief states in search by considering AND – OR search tree structure and solution is a connected portion of the AND- OR tree.

If a belief state has proved as unsolvable, any superset is unsolvable as well and marked as unsolvable. It allows cut off the tree branches which has failed belief states supersets when it expands.

2.



Let s = state

Loop :

If s = sg then stop

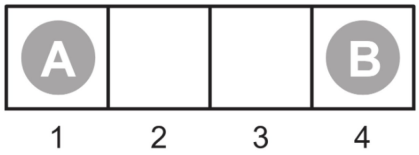
If s = s0 then act a1 and go loop

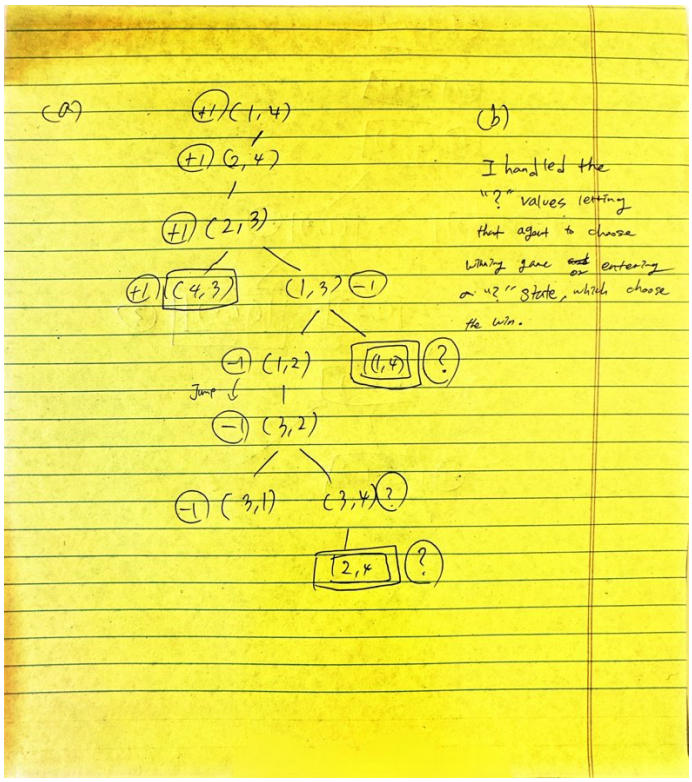
Else if s = s1 then act a3 and go loop

Else if s = s2 then act a4 and go loop

Else if s = s3 then act a5 and go loop

3.(a) & (b)





(c)
A standard minimax algorithm search is depth-first and it will enter an infinite loop in the cycle. I can fix it by checking if current state is already on stack. If is already on the stack, return "?" value by propagating it up, which means loop. Other good methods like strongly connected component analysis can determine well-defined values for this cycles

(d)
On base cases, n =3, which is odd lose of A because A moves from 1 to 2 and B jumps from 3 to 1 and n=4, which is even is lose of B. For n > 4, initial moves are same since both of them step toward.
If n -2 is win for A, A reaches to square n -1 before B reaches to square 2. Thus, A finally reaches n before B reaches 1. A win.
If n -2 is win for B, B gets the advantage by symmetry and A must lose game again.
Thus, A wins when n = even and lose when n = odd.